

Лабораторная работа №5:
Исследование задачи обедающих
философов средствами сетей Петри
Имитационное моделирование

true

today

Информация

Докладчик

- Авдадаев Джамал Геланиевич
- Российский университет дружбы народов им. П. Лумумбы
- 1032230169@rudn.ru

Вводная часть

Цель работы

- Реализовать сети Петри для задачи обедающих философов: классическую и с арбитром
- Научиться обнаруживать тупики (deadlock) в модели
- Визуализировать динамику маркировки и провести параметрическое исследование

Объект и предмет исследования

- **Объект:** сеть Петри для задачи обедающих философов
- **Предмет:** условия возникновения тупика и динамика маркировки в классической сети и сети с арбитром

Математическая постановка

- Сеть Петри: позиции Think_i , Hungry_i , Eat_i , Fork_i

- Матрица инцидентности задаёт переходы между состояниями философов и вилок
- Классическая сеть допускает одновременный захват соседних вилок → возможен deadlock
- Сеть с арбитром ограничивает число одновременно «голодных» философов

Используемые инструменты

- Julia 1.12.6, DrWatson v2.19.1
- OrdinaryDiffEq, Plots v1.41.6, DataFrames v1.8.2, CSV v0.10.16
- Literate.jl для литературного программирования
- Quarto / Jupyter Notebook

Реализация модели

Модуль DiningPhilosophers

Структура PetriNet и экспортируемые функции модуля

Figure 1: Структура PetriNet и экспортируемые функции модуля

- PetriNet: places, transitions, матрица инцидентности
- Функции: build_classical_network, build_arbiter_network, simulate_stochastic, detect_deadlock, plot_marking_evolution

Базовый скрипт: N = 5

dining_philosophers.jl: базовый эксперимент

Figure 2: dining_philosophers.jl: базовый эксперимент

- Классическая сеть и сеть с арбитром, N = 5, tmax = 50.0
- Результаты → dining_classic.csv, dining_arbiter.csv
- Проверка detect_deadlock для обоих вариантов

Базовый эксперимент (N = 5)

Результат: deadlock в классической сети

- Классическая сеть: Deadlock обнаружен: true
- Сеть с арбитром: Deadlock обнаружен: false

Deadlock обнаружен: true (классика) / false (арбитр)

Figure 3: Deadlock обнаружен: true (классика) / false (арбитр)

Классическая сеть: динамика маркировки

- Моделирование останавливается на $t \approx 2.6$
- Поест удается только философ 3 (два коротких импульса)
- К $t \approx 1.0$ четыре вилки (Fork_1–Fork_4) зафиксированы на 0

Сеть с арбитром: динамика маркировки

- Моделирование идёт весь интервал $t \in [0, 50]$
- Все 5 философов регулярно переходят в состояние «ест»
- Вилки активно переключаются — нет захваченных навсегда ресурсов

Итоговый сравнительный график

- Классика ($t \in [0, 2.5]$): только 2 импульса еды, затем тупик
- Арбитр ($t \in [0, 50]$): регулярные импульсы еды у всех 5 философов

Анимация и литературное программирование

Анимация маркировки (N = 3)

- $N = 3$, $t_{\max} = 30.0$, `Random.seed!(123)`
- Начальное состояние: все философы Think, все вилки свободны (Fork_1–3 = 1)
- Сохранено в `philosophers_simulation.gif`, `fps = 5`

Литературные версии скриптов

- Преобразованы: базовый скрипт, анимация, параметрическое исследование, отчёт
- Сгенерированы Jupyter Notebook (.ipynb) и Quarto-документы (.qmd / .html)

Сгенерированные Notebook и Quarto-документы

- 4 скрипта $\rightarrow$ 4 .ipynb + 4 .qmd/.html
- Все документы успешно сгенерированы и выполнены

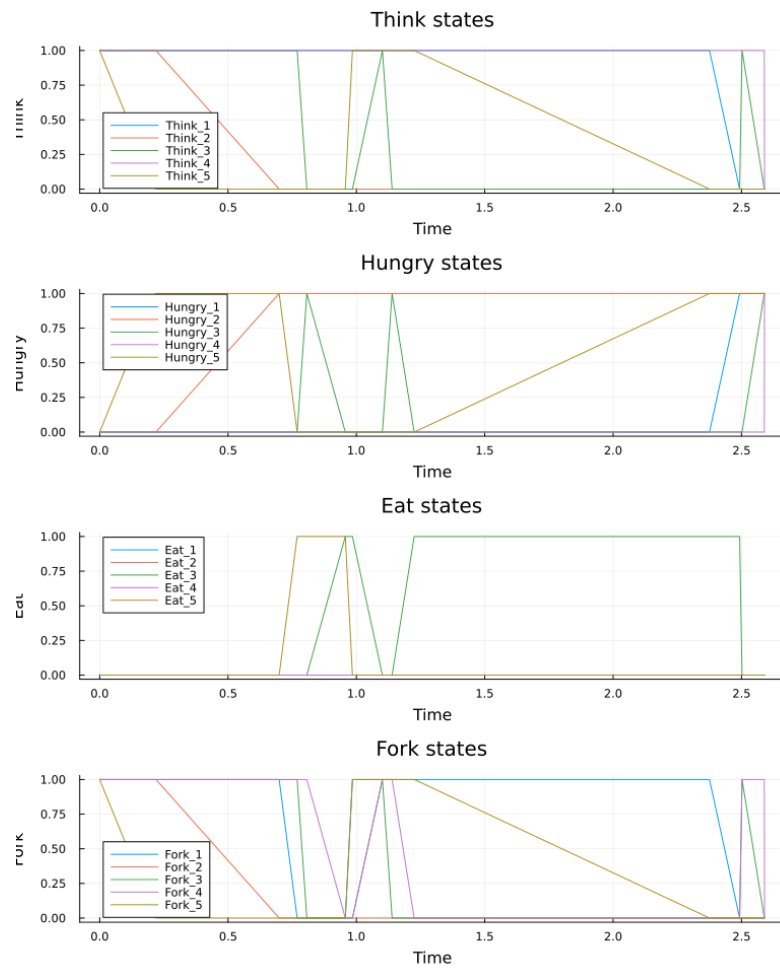


Figure 4: classic_simulation.png

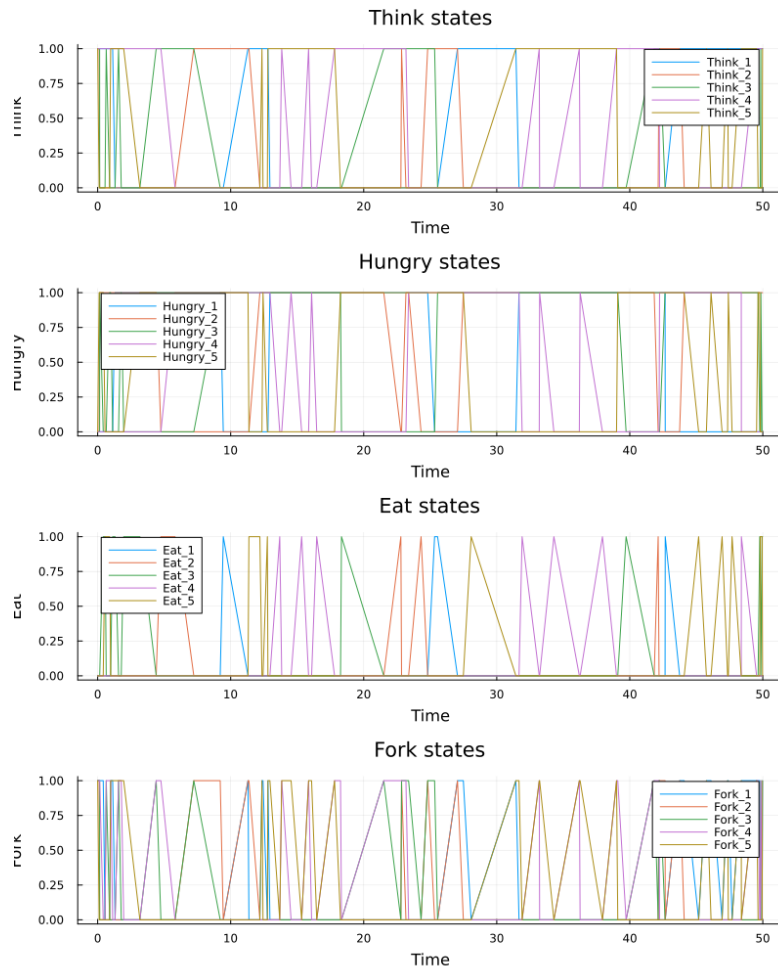


Figure 5: arbiter_simulation.png

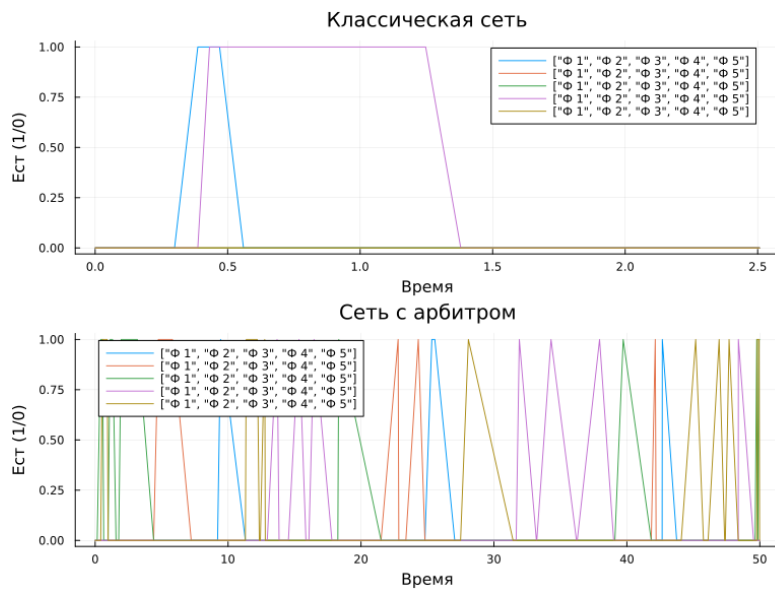


Figure 6: final_report.png: состояния «ест» для классики и арбитра

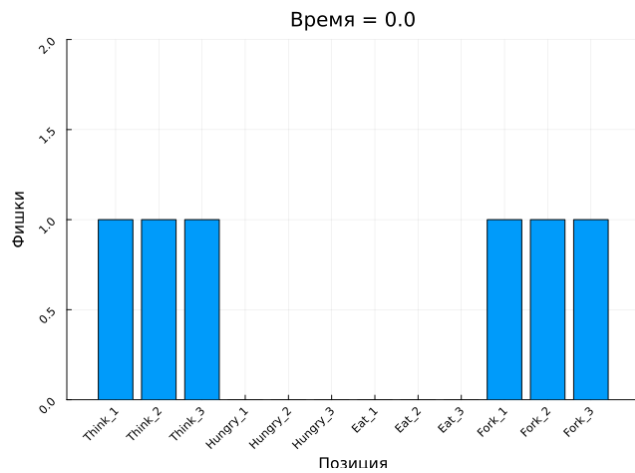


Figure 7: philosophers_simulation.gif: начальный кадр, Время = 0.0

dining_philosophers_literate.jl

Figure 8: dining_philosophers_literate.jl

Literate.notebook и Literate.markdown с QuartoFlavor

Figure 9: Literate.notebook и Literate.markdown с QuartoFlavor

Параметрическое исследование

Набор параметров

dining_philosophers_parametrized_literate.jl

Figure 10: dining_philosophers_parametrized_literate.jl

- param_sets: (N=3, tmax=30.0), (N=5, tmax=50.0), (N=7, tmax=70.0)
- Функция run_experiment(N, tmax) строит обе сети и проверяет deadlock

Результаты исследования

Таблица deadlock_classic / deadlock_arbiter для N = 3, 5, 7

Figure 11: Таблица deadlock_classic / deadlock_arbiter для N = 3, 5, 7

- Для всех N (3, 5, 7): deadlock_classic = true
- Для всех N (3, 5, 7): deadlock_arbiter = false

График: классическая сеть, N = 3, 5, 7

- N = 3 → тупик к $t \approx 0.7$ (1 импульс еды)
- N = 5 → тупик к $t \approx 17.5$ (несколько импульсов еды)
- N = 7 → тупик к $t \approx 6.5$ (2 импульса еды)

Заключение и выводы

Итоги работы

- Реализованы классическая сеть и сеть с арбитром для задачи обедающих философов
- Для N = 5 классическая сеть приходит к тупику ($t \approx 2.6$), сеть с арбитром работает без тупика на $t \in [0, 50]$
- Для N = 3, 5, 7 классическая сеть всегда приходит к тупику, арбитр — никогда

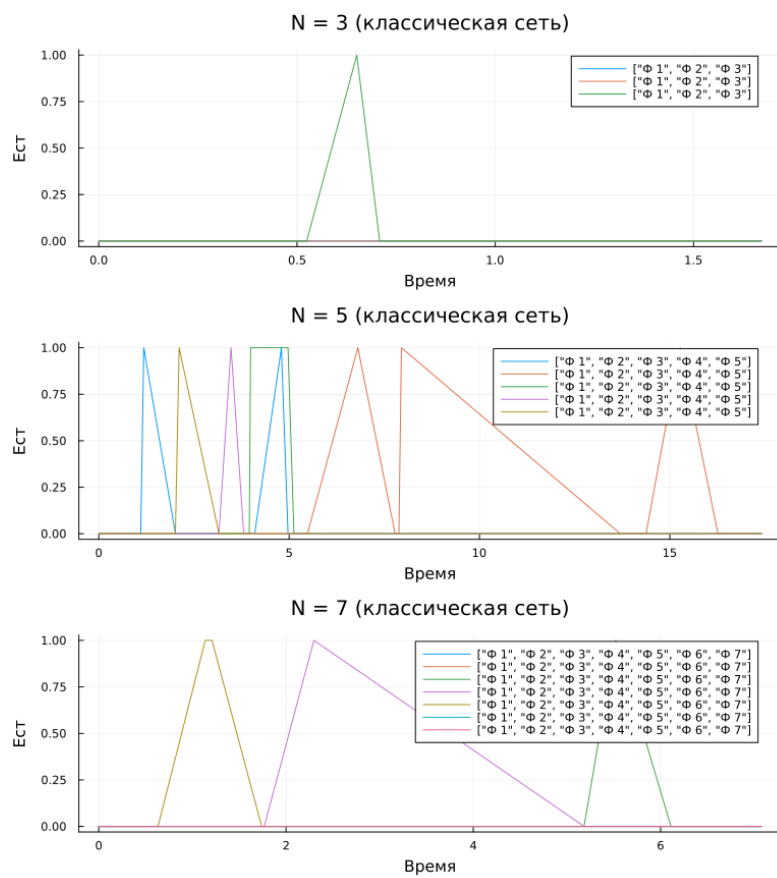


Figure 12: parameter_comparison.png

- Время до тупика в классической сети немонотонно: $t \approx 0.7$ ($N=3$), $t \approx 17.5$ ($N=5$), $t \approx 6.5$ ($N=7$)
- Построена анимация маркировки сети для $N = 3$

Освоенные навыки

- Реализация сетей Петри и проверка тупиковых ситуаций на Julia
- Стохастическое моделирование и сохранение результатов в CSV
- Построение анимаций (@animate, gif)
- Литературное программирование: Literate.jl → Jupyter Notebook и Quarto (QuartoFlavor)